第 10 章 哈希表和 unordered_set 与 unordered_map

1. 哈希表的概念

（可以用 ppt 演示）

1.1 哈希表的定义

哈希表(hash table)，又称散列表，是根据关键字直接进行访问的数据结构。

哈希表建立了一种关键字和存储地址之间的直接映射关系，使每个关键字与结构中的唯一存储位置相对应。理想情况下，在散列表中进行查找的时间复杂度为 $O(1)$，即与表中的元素数量无关。因此哈希表是一种存储和查找非常快的结构。

1.2 哈希函数

将关键字映射成对应的地址的函数就是**哈希函数**，也叫作散列函数，记为 `Hash(key) = Addr`。

哈希函数的本质也是一个函数，它的作用是，你给它一个关键字，它给你一个该关键字对应的存储位置。

1.3 哈希冲突

哈希函数可能会把两个或两个以上的不同关键字映射到同一地址，这种情况称为哈希冲突，也称散列冲突。起冲突的不同关键字，称它们为同义词。

由此可见，设计一个优秀的哈希表，不仅需要需要设计一个好的哈希函数，也要能够处理哈希冲突。那么，学习哈希表的重点就是**设计哈希函数**和**处理哈希冲突**。

2. 常见的哈希函数

2.1 直接定址法

第一个案例中，统计字符串中，小写字符出现的次数使用的方法，就是**直接定址法**。

直接取关键字的某个线性函数值为散列地址，散列函数是 hash(key) = key 或 hash(key) = a× key + b

其中 a 与 b 为常数。这种方式计算比较简单，适合关键字的分布基本连续的情况，但是若关键字分布不连续，空位较多，则会造成存储空间的浪费。

2.2 除留余数法

哈希冲突那里的案例，所用的哈希函数就是除留余数法。

除留余数法，顾名思义，假设哈希表的大小为 M，那么通过 key 除以 M 的余数作为映射位置的下标，也就是哈希函数为：hash(key) = key % M。因此，这种方法的重点就是选好模数 M。

- 建议 M 取不太接近 2 的整数次冥的一个质数(素数)。（具体原理可以参考算法导论里面的证明，选素数可以让分布更均匀，进而减少冲突。）

2.3 其他方法

上面的两种方法是《算法导论》书籍中讲解的方法，除此之外还有**乘法散列法**和**全域散列法**。

《殷人昆 数据结构：用面向对象方法与C++语言描述 （第二版）》和 《[数据结构(C语言版)].严蔚敏_吴伟民》等教材型书籍上面还给出了**平方取中法、折叠法、随机数法、数学分析法**等，这些方法相对更适用于一些局限的特定场景，有兴趣可以去看看这些书籍。

3. 处理哈希冲突

有时候哈希表无论选择什么哈希函数都无法避免冲突，那么插入数据时，如何解决冲突呢？主要有两种方法，**线性探测法**和**链地址法**。

（可以用 ppt 演示）

3.1 线性探测法

从发生冲突的位置开始，依次线性向后探测，直到寻找到下一个没有存储数据的位置为止，如果走到哈希表尾，则回绕到哈希表头的位置。

3.2 链地址法

链地址法中所有的数据不再直接存储在哈希表中，哈希表中存储一个指针，没有数据映射这个位置时，这个指针为空，有多个数据映射到这个位置时，我们把这些冲突的数据链接成一个链表，挂在哈希表这个位置下面。

4. 哈希表的模拟实现

这里选用的哈希函数就是除留余数法。

案例：

题目描述：

> 维护一个数据结构，初始时为空。请支持下面的操作：
>
> - `1 x`：插入元素 x；
> - `2 x`：查询 x 是否在数据结构中。
>
> 现有 n 次操作，针对每次查询，输出是否在该数据结构中。

输入描述：

> 第一行一个整数 n，表示查询次数。
>
> 之后 n 行，第 i 行两个整数 op、x，分别表示第 op 个操作，以及元素 x。

测试用例：

```
测试：
12
1 1
1 2
1 3
1 4
1 5
2 2
1 6
2 5
1 7
2 8
2 4
2 25
```

4.1 线性探测法

4.1.1 创建

h	INF	INF	INF	INF	INF	INF	INF
下标	0	1	2	3	4	5	6

```cpp
#include <iostream>
#include <cstring>

using namespace std;

// N 是质数
const int N = 23, INF = 0x3f3f3f3f;

int h[N]; // 哈希表

// 先把哈希表中所有元素初始化成一个不会出现的值
void init()
{
    memset(h, 0x3f, sizeof h);
}

int main()
{
    init();

    return 0;
}
```

4.1.2 哈希函数以及处理哈希冲突

除留余数法：`hash(key) = key % N`

但是要注意，`key` 有可能是负数，取模之后会变成负数。

- 负数补正的操作为：加上模数即可。
- 但是正数加上模数会变大，所以统一再取一次模。

最终就是 `(key % N + N) % N`，简称 模加模

```cpp
// 哈希函数 f(x) 返回 x 映射的位置
int f(int x)
```

```
{
    // 经典操作：模 加 模
    int idx = (x % N + N) % N; // 为了避免出现负数

    // 处理冲突
    while(h[idx] != INF && h[idx] != x)
    {
        idx++; // 线性探测
        if(idx == N) idx = 0; // 如果走到头了，就拐个弯
    }
    return idx;
}
```

4.1.3 添加元素

> 通过哈希函数找到合适的位置，然后放上去即可

```
// 添加元素
void insert(int x)
{
    int idx = f(x); // 哈希函数计算位置
    h[idx] = x;
}
```

4.1.4 查找元素

> 通过哈希函数找到映射位置，看看里面的值是不是 x

```
// 查找元素
bool find(int x)
{
    int idx = f(x);
    return h[idx] == x;
}
```

4.1.5 所有测试代码

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 23, INF = 0x3f3f3f3f;

int h[N]; // 哈希表

// 初始化
void init()
{
    memset(h, 0x3f, sizeof h);
}

// 哈希函数 - 计算出 x 的存储位置
int f(int x)
{
    int id = (x % N + N) % N;

    // 处理哈希冲突 - 线性探测法
    while(h[id] != INF && h[id] != x)
    {
        id++;
        if(id == N) id = 0; // 如果走到尾部，从头开始
    }

    return id;
}

// 添加元素
void insert(int x)
{
    int id = f(x);
    h[id] = x;
}

// 查找元素
bool find(int x)
{
    int id = f(x);
    return h[id] == x;
}

// 删除元素

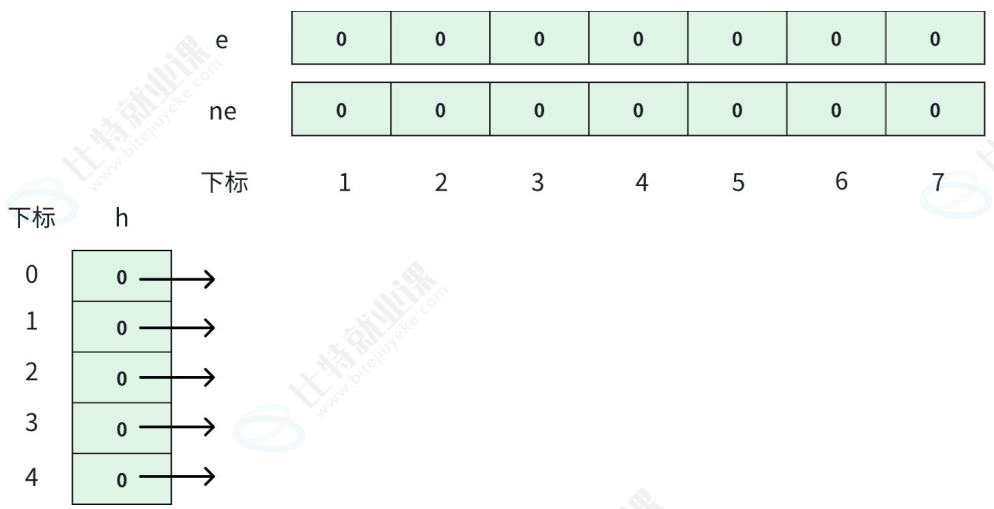
int main()
```

```
48  {
49      init();
50      int n; cin >> n;
51
52      while(n--)
53      {
54          int op, x; cin >> op >> x;
55          if(op == 1) // 插入
56          {
57              insert(x);
58          }
59          else // 查询
60          {
61              if(find(x)) cout << "yes" << endl;
62              else cout << "no" << endl;
63          }
64      }
65
66
67      return 0;
68  }
```

4.2 链地址法

4.2.1 创建

> 实现方式与**树的链式向前星**一模一样~
>
> 本质就是用数组模拟链表。



```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 23;

int h[N]; // 哈希表
int e[N], ne[N], id; // 链表里面的一个一个结点
```

4.2.2 哈希函数

> 除留余数法

```cpp
// 哈希函数
int f(int x)
{
    return (x % N + N) % N;
}
```

4.2.3 查找元素

1. 先计算 x 对应的哈希值；
2. 在哈希值所在的链表中查找。

```cpp
// 查找元素
bool find(int x)
{
    int idx = f(x); // 哈希值
    // 遍历 idx 的链表
    for(int i = h[idx]; i; i = ne[i])
    {
        if(e[i] == x) return true;
    }
    return false;
}
```

4.2.4 添加元素以及处理哈希冲突

1. 先判断是否已经在哈希表中；
2. 如果不在，就头插在哈希值所对应的链表后。

```
// 添加元素+哈希冲突
void insert(int x)
{
    if(find(x)) return; // 如果找到了，就不添加了

    int idx = f(x);
    // 头插在 idx 后面
    id++;
    e[id] = x;
    ne[id] = h[idx];
    h[idx] = id;
}
```

4.2.5 所有测试代码

```
#include <iostream>

using namespace std;

const int N = 23;

int h[N]; // 哈希表
int e[N], ne[N], id;

// 哈希函数
int f(int x)
{
    return (x % N + N) % N;
}

// 插入元素 - 处理哈希冲突
void insert(int x)
{
    int idx = f(x);

    // 把 x 头插到 idx 所在的链表中
    id++;
```

```
23      e[id] = x;
24
25      // 链地址法
26      ne[id] = h[idx];
27      h[idx] = id;
28  }
29
30  // 查找元素
31  bool find(int x)
32  {
33      int idx = f(x);
34
35      for(int i = h[idx]; i; i = ne[i])
36      {
37          if(e[i] == x) return true;
38      }
39
40      return false;
41  }
42
43  int main()
44  {
45      int n; cin >> n;
46
47      while(n--)
48      {
49          int op, x; cin >> op >> x;
50          if(op == 1) // 插入
51          {
52              insert(x);
53          }
54          else // 查询
55          {
56              if(find(x)) cout << "yes" << endl;
57              else cout << "no" << endl;
58          }
59      }
60
61
62      return 0;
63  }
```

5. unordered_set / unordered_multiset

> set 与 unordered_set 的区别就是，前者是用**红黑树**实现的，后者是用**哈希表**实现的。使用的方式是完全一样的。无非就是**存储和查找的效率不一样**，以及前者存的是**有序的，后者无序**。

5.1 创建 unordered_set

```cpp
#include <iostream>
#include <unordered_set>

using namespace std;

int main()
{
    unordered_set <int> mp1;
    unordered_set <string> mp2;

    return 0;
}
```

5.2 size / empty

1. `size`：求哈希表内实际元素的个数。时间复杂度为：$O(1)$。
2. `empty`：判断哈希表是否为空。时间复杂度为：$O(1)$。

5.3 begin / end

> 迭代器，可以使用范围 for 遍历哈希表中的所有元素。
>
> 但是要注意，哈希表不同于红黑树，遍历出来的结果是无序的。

5.4 insert

> 往哈希表中插入一个元素。
>
> 时间复杂度可以近似认为：$O(1)$。

5.5 erase

> 删除一个元素。
>
> 时间复杂度可以近似认为：$O(1)$。

5.6 find / count

1. `find`：查找一个元素，返回的是迭代器。时间复杂度可以近似认为：$O(1)$。
2. `count`：查询元素出现的次数，一般用来判断元素是否在哈希表中。时间复杂度可以近似认为：$O(1)$。

5.7 所有测试代码

```cpp
#include <iostream>
#include <unordered_set>

using namespace std;

int main()
{
    string strs[] = {"张三", "李四", "王五", "赵六", "小杨", "小珂"};

    unordered_set<string> mp;

    for(auto& s : strs)
    {
        mp.insert(s); // 插入
    }

    if(mp.count("张三")) cout << "yes" << endl;
    else cout << "no" << endl;

    mp.erase("张三");

    if(mp.count("张三")) cout << "yes" << endl;
    else cout << "no" << endl;

    for(auto& s : mp)
    {
        cout << s << " ";
    }

    return 0;
}
```

6. unordered_map / unordered_multimap

6.1 创建 unordered_map

```cpp
#include <iostream>
#include <vector>
#include <unordered_map>

using namespace std;

int main()
{
    unordered_map<int, int> mp1;
    unordered_map<int, string> mp2;
    unordered_map<string, int> mp3;
    unordered_map<int, vector<int>> mp4; // 这个是不是可以存图?

    return 0;
}
```

6.2 size / empty

1. `size`：求哈希表中实际元素的个数。时间复杂度：$O(1)$。
2. `empty`：判断哈希表是否为空。时间复杂度：$O(1)$。

6.3 begin / end

迭代器，可以使用范围 for 遍历整个哈希表。

6.4 insert

向哈希表中插入一个元素。

时间复杂度可以近似认为：$O(1)$。

6.5 operator[]

重载 []，使的 umordered_map 可以像数组一样使用。

6.6 erase

> 删除一个元素
>
> 时间复杂度可以近似认为：$O(1)$。

6.7 find / count

1. `find`：查找一个元素，返回的是迭代器。时间复杂度可以近似认为：$O(1)$。
2. `count`：查询元素出现的次数，一般用来判断元素是否在哈希表中。时间复杂度可以近似认为：$O(1)$。

6.8 所有测试代码

```cpp
#include <iostream>
#include <vector>
#include <unordered_map>

using namespace std;

void test()
{
    unordered_map<int, vector<int>> mp;
    mp[1].push_back(2);
    mp[2] = {3, 4, 5};
    mp[3].push_back(1);
    mp[3].push_back(2);
    mp[3].push_back(3);

    for(auto& [a, v] : mp)
    {
        cout << a << ": ";
        for(auto b : v) cout << b << " ";
        cout << endl;
    }
}

int main()
{
    test();
    unordered_map<string, int> mp;
```

```cpp
    // 写一个大括号，把需要放进去的元素括起来即可
    mp.insert({"张三", 1});
    mp.insert({"李四", 2});
    mp.insert({"王五", 3});

    // operator[]: 可以让 map 像数组一样使用
    // 赋值
    mp["赵六"] = 4; // 相当于往 mp 里面插入一个 <"赵六", 4>


    // 查询里面的值
    if(mp["赵六"] == 4) cout << "yes" << endl;
    else cout << "no" << endl;
    // 使用查询的时候要注意，如果 map 中本身没有该元素，它会先插入，然后再拿值
    // 插入的时候，第二个关键字是默认值
    // 如果是数，那就是 0
    // 如果是字符串，那就是空串

    if(mp["小美"]) cout << "no" << endl; // 会把 <"小美", 0> 放进去

    if(mp.count("小帅")) cout << "yes" << endl;
    else cout << "no" << endl;

    mp.erase("小美");
    for(auto& [s, num] : mp)
    {
        cout << s << "编号为:  " << num << endl;
    }

    return 0;
}
```

7. 哈希表算法题

7.1 学籍管理

> **题目来源：** 洛谷
>
> **题目链接：** 【深基17.例6】学籍管理
>
> **难度系数：** ★

【题目描述】

> 您要设计一个学籍管理系统，最开始学籍数据是空的，然后该系统能够支持下面的操作（不超过 10^5 条）：
>
> - 插入与修改，格式 `1 NAME SCORE`：在系统中插入姓名为 NAME(由字母和数字组成不超过 20 个字符的字符串，区分大小写)，分数为 SCORESCORE（ $0 < SCORE < 2^{31}$ ） 的学生。如果已经有同名的学生则更新这名学生的成绩为 SCORE。如果成功插入或者修改则输出 `OK`。
> - 查询，格式 `2 NAME`：在系统中查询姓名为 NAME 的学生的成绩。如果没能找到这名学生则输出 `Not found`，否则输出该生成绩。
> - 删除，格式 `3 NAME`：在系统中删除姓名为 NAME 的学生信息。如果没能找到这名学生则输出 `Not found`，否则输出 `Deleted successfully`。
> - 汇总，格式 `4`：输出系统中学生数量。

【示例一】

> 输入：
>
> 5
>
> 1 lxl 10
>
> 2 lxl
>
> 3 lxl
>
> 2 lxl
>
> 4
>
> 输出：
>
> OK
>
> 10
>
> Deleted successfully
>
> Not found
>
> 0

【解法】

用 `unordered_map` 维护学生信息即可。

【参考代码】

```cpp
#include <iostream>
#include <unordered_map>

using namespace std;
```

```cpp

int main()
{
    unordered_map<string, int> mp;

    int T; cin >> T;

    while(T--)
    {
        int op;
        string name;
        cin >> op;
        if(op == 1) // 插入 + 修改
        {
            cin >> name;
            int x; cin >> x;
            mp[name] = x;
            cout << "OK" << endl;
        }
        else if(op == 2) // 查询
        {
            cin >> name;
            if(mp.count(name)) cout << mp[name] << endl;
            else cout << "Not found" << endl;
        }
        else if(op == 3) // 删除
        {
            cin >> name;
            if(mp.count(name))
            {
                mp.erase(name); // 删除
                cout << "Deleted successfully" << endl;
            }
            else cout << "Not found" << endl;
        }
        else
        {
            cout << mp.size() << endl;
        }
    }

    return 0;
}
```

7.2 不重复数字

> **题目来源：** 洛谷
>
> **题目链接：** P4305 [JLOI2011] 不重复数字
>
> **难度系数：** ★

【题目描述】

> 给定 n 个数，要求把其中重复的去掉，只保留第一次出现的数。

【输入描述】

> **本题有多组数据。**
>
> 第一行一个整数 T，表示数据组数。
>
> 对于每组数据：
>
> 第一行一个整数 n。
>
> 第二行 n 个数，表示给定的数。
>
> 对于 100% 的数据，$1 \leq T \leq 50$，$1 \leq n \leq 5 \times 10^4$，给出的数在 32 位有符号整数范围内。

【输出描述】

> 对于每组数据，输出一行，为去重后剩下的数，两个数之间用一个空格隔开。

【示例一】

> 输入：
>
> ```
> 2
> 11
> 1 2 18 3 3 19 2 3 6 5 4
> 6
> 1 2 3 4 5 6
> ```
>
> 输出：
>
> ```
> 1 2 18 3 19 6 5 4
> 1 2 3 4 5 6
> ```

【解法】

创建一个哈希表用来标记哪些数已经出现过。

在扫描整个数组的过程中，判断该数是否已经在哈希表中：

- 如果在哈希表中：不做处理；

- 如果不在哈希表中：输出这个数，然后加入到哈希表中。

【参考代码】

```cpp
#include <iostream>
#include <unordered_set>

using namespace std;

int main()
{
    int T; // cin >> T;
    scanf("%d", &T);

    while(T--)
    {
        int n; // cin >> n;
        scanf("%d", &n);
        unordered_set<int> mp;

        for(int i = 1; i <= n; i++)
        {
            int x; // cin >> x;
            scanf("%d", &x);
            if(!mp.count(x)) // 是第一次出现
            {
                // cout << x << " ";
                printf("%d ", x);
                mp.insert(x);
            }
        }
        // cout << endl;
        puts("");
    }

    return 0;
}
```

7.3 阅读理解

> **题目来源：** 洛谷
>
> **题目链接：** P3879 [TJOI2010] 阅读理解

难度系数： ★

【题目描述】

英语老师留了 N 篇阅读理解作业，但是每篇英文短文都有很多生词需要查字典，为了节约时间，现在要做个统计，算一算某些生词都在哪几篇短文中出现过。

【输入描述】

第一行为整数 N，表示短文篇数，其中每篇短文只含空格和小写字母。

按下来的 N 行，每行描述一篇短文。每行的开头是一个整数 L，表示这篇短文由 L 个单词组成。接下来是 L 个单词，单词之间用一个空格分隔。

然后为一个整数 M，表示要做几次询问。后面有 M 行，每行表示一个要统计的生词。

对于 100% 的数据，$1 \leq M \leq 10^4$，$1 \leq N \leq 10^3$。

每篇短文长度（含相邻单词之间的空格）$\leq 5 \times 10^3$ 字符，每个单词长度 ≤ 20 字符。

【输出描述】

对于每个生词输出一行，统计其在哪几篇短文中出现过，并按从小到大输出短文的序号，序号不应有重复，序号之间用一个空格隔开（注意第一个序号的前面和最后一个序号的后面不应有空格）。如果该单词一直没出现过，则输出一个空行。

【示例一】

输入：

3

9 you are a good boy ha ha o yeah

13 o my god you like bleach naruto one piece and so do i

11 but i do not think you will get all the points

5

you

i

o

all

naruto

输出：

1 2 3

2 3

1 2

```
3

2
```

【解法】

为每一个单词创建一个 `set`，标记每个单词在哪一篇文章中出现过。

【参考代码】

```cpp
#include <iostream>
#include <set>
#include <unordered_map>

using namespace std;

unordered_map<string, set<int>> mp; // 标记单词在哪些文章中出现过

int main()
{
    int n; cin >> n;
    for(int i = 1; i <= n; i++)
    {
        int l; cin >> l;
        while(l--)
        {
            string s; cin >> s;
            mp[s].insert(i);
        }
    }

    int m; cin >> m;
    while(m--)
    {
        string s; cin >> s;
        for(auto i : mp[s])
        {
            cout << i << " ";
        }
        cout << endl;
    }

    return 0;
}
```

7.4 A - B 数对

> **题目来源：** 洛谷
>
> **题目链接：** P1102 A-B 数对
>
> **难度系数：** ★

【题目描述】

> 给出一串正整数数列以及一个正整数 C，要求计算出所有满足 $A-B=C$ 的数对的个数（不同位置的数字一样的数对算不同的数对）。

【输入描述】

> 输入共两行。
>
> 第一行，两个正整数 N, C。
>
> 第二行， N 个正整数，作为要求处理的那串数。
>
> 对于 100% 的数据，$1 \leq N \leq 2 \times 10^5$ ， $0 \leq a_i < 2^{30}$ ， $1 \leq C < 2^{30}$ 。

【输出描述】

> 一行，表示该串正整数中包含的满足 $A-B=C$ 的数对的个数。

【示例一】

> 输入：
>
> ```
> 4 1
> 1 1 2 3
> ```
>
> 输出：
>
> ```
> 3
> ```

【解法】

创建一个哈希表，统计一下所有数字出现的次数。

然后依次遍历数组中的每一个数 $a[i]$，统计一下 $b=a[i]-c$ 这个数出现的次数。

【参考代码】

```cpp
#include <iostream>
#include <unordered_map>

using namespace std;
```

```cpp

typedef long long LL;
const int N = 2e5 + 10;

LL n, c;
LL a[N];
unordered_map<int, int> mp; // <数，该数出现的次数>

int main()
{
    cin >> n >> c;
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i];
        mp[a[i]]++;
    }

    LL ret = 0;
    for(int i = 1; i <= n; i++)
    {
        // b = a[i]
        // 找 c + a[i]
        ret += mp[c + a[i]];
    }

    cout << ret << endl;

    return 0;
}
```

7.5 Cities and States

> **题目来源：** 洛谷
>
> **题目链接：** P3405 [USACO16DEC] Cities and States S
>
> **难度系数：** ★★

【题目描述】

> Farmer John 有若干头奶牛。为了训练奶牛们的智力，Farmer John 在谷仓的墙上放了一张美国地图。地图上表明了每个城市及其所在州的代码（前两位大写字母）。
>
> 由于奶牛在谷仓里花了很多时间看这张地图，他们开始注意到一些奇怪的关系。例如，`FLINT` 的前两个字母就是 `MIAMI` 所在的 `FL` 州，`MIAMI` 的前两个字母则是 `FLINT` 所在的 `MI` 州。确切地说，对于两个城市，它们的前两个字母互为对方所在州的名称。

我们称两个城市是一个一对「特殊」的城市，如果他们具有上面的特性，并且来自不同的州。对于总共 N 座城市，奶牛想知道有多少对「特殊」的城市存在。请帮助他们解决这个有趣的地理难题！

【输入描述】

输入共 $N+1$ 行。

第一行一个正整数 N，表示地图上的城市的个数。
接下来 N 行，每行两个字符串，分别表示一个城市的名称（$2 \sim 10$ 个大写字母）和所在州的代码（2 个大写字母）。同一个州内不会有两个同名的城市。

对于 100% 的数据，$1 \leq N \leq 2 \times 10^5$，城市名称长度不超过 10。

【输出描述】

输出共一行一个整数，代表特殊的城市对数。

【示例一】

输入：

```
6
MIAMI FL
DALLAS TX
FLINT MI
CLEMSON SC
BOSTON MA
ORLANDO FL
```

输出：

```
1
```

【解法】

解法：将映射关系存成一个字符串。

【参考代码】

```cpp
#include <iostream>
#include <unordered_map>

using namespace std;

```

```cpp
int n;

int main()
{
    cin >> n;
    unordered_map<string, int> mp; // <拼接后的对应关系，次数>int ret = 0;
    while(n--)
    {
        string a, b; cin >> a >> b;
        a = a.substr(0, 2);

        if(a == b) continue; // 往后查找的时候属于都一个州

        ret += mp[b + a]; // 统计 b->a 一共有多少个
        mp[a + b]++;
    }
    cout << ret << endl;

    return 0;
}
```